

MULTIVIEW: A Portable C99 System for Two-Camera Scene Analysis and 3D Model Extraction

A self-contained overview of the system and its mathematics

Project documentation — v0.2.0

August 2, 2026

Abstract

MULTIVIEW is a small, dependency-free C99 library for metric scene analysis and 3D model extraction from the synchronized video streams of *two stationary cameras* observing an overlapping volume, optionally augmented by a collection of still photographs and, in a later stage, range-finder measurements. Calibration is part of the system: the rig is calibrated from views of a printed letter-page checkerboard of known square size, or from a computer display under our control showing synchronized Gray-code patterns. This document doubles as a self-study overview of the underlying mathematics: each concept is developed from first principles, cross-linked to the modules that implement it, illustrated by a *hand-computable* worked example, and sourced in the bibliography. All quantitative claims ([section 13](#)) are reproducible from the deterministic test programs in the repository.

Contents

1	Introduction and capabilities	2
2	System overview	3
3	Mathematical toolkit	4
3.1	Homogeneous coordinates	4
3.2	Rotations	4
3.3	The cross-product matrix	4
3.4	Homogeneous least squares and the SVD	4
4	Camera model	5
5	Calibration from planar targets	5
5.1	The homography of a plane	5
5.2	Zhang’s constraints and the closed-form intrinsics	6
5.3	Radial distortion and alternation	6
5.4	The printed target	7
5.5	The display as an active target	7
5.6	Design: the rotating display	7
6	Two-view epipolar geometry	9

7	Triangulation	10
8	Rectification and dense stereo	10
9	From depth to models	11
10	Scaling the rig: more cameras and more pixels	11
10.1	<i>N</i> cameras: accuracy	11
10.2	<i>N</i> cameras: occlusion and topology	12
10.3	Resolution	13
10.4	Unequal resolutions	13
11	Planned extensions	14
11.1	Auxiliary photo collections	14
11.2	Range-finder fusion (optional, kept open)	14
12	Numerical implementation notes	15
13	Basic results	16
14	Roadmap	17
A	Camera hardware and the quality of data	17
A.1	Shutter: global vs rolling	17
A.2	Synchronization: hardware trigger vs free-running	18
A.3	Interface: UVC vs machine-vision stacks	18
A.4	Optics: M12 vs C/CS mounts	18
A.5	Monochrome vs color	19
A.6	Summary and candidate hardware	19
A.7	Blending heterogeneous cameras	20

1 Introduction and capabilities

The target deployment is deliberately narrow and fully under our control: two cameras are rigidly mounted and observe the same working volume. The system first *calibrates itself* from views of a known planar target ([section 5](#)); thereafter video is processed frame by frame. Because the cameras do not move, every geometric quantity that depends only on the rig — projection matrices, the fundamental matrix, the rectifying homographies — is computed *once* and reused for every frame, which is the central efficiency win of the stationary-rig assumption.

The implemented core (v0.2.0) provides:

1. plane-based calibration of intrinsics, distortion, and pose from a printed letter-page checker-board or from a controlled display showing Gray-code patterns ([section 5](#));
2. a calibrated pinhole camera model with Brown radial-tangential lens distortion, forward projection and ray back-projection ([section 4](#));
3. exact two-view epipolar geometry from the calibration, and the normalized 8-point algorithm to estimate or verify it from image correspondences ([section 6](#));

4. N -view linear triangulation of corresponding image points into metric 3D, with reprojection-error reporting (section 7);
5. planar stereo rectification of the pair, and dense correspondence by block matching with sub-pixel refinement, giving per-pixel depth (section 8);
6. point-cloud assembly and PLY export for inspection in standard mesh tools, plus minimal PGM image I/O (section 9).

Planned extensions — structure-from-motion over an auxiliary photo collection and fusion of range-finder data — are designed but not yet implemented; their theory and integration points are described in section 11 so that the code can grow into them without rearchitecting.

How to read this document. section 3 collects the small amount of linear algebra everything else stands on. section 4–section 8 then follow the data through the system: camera model, calibration, two-view geometry, triangulation, dense stereo. Every section ends in a worked example with numbers chosen so the computation fits on paper; doing them by hand *is* the intended self-study exercise, and each can then be checked against the library.

Design constraints. The library is strict C99 with no dependencies beyond `libm`, row-major `double` arrays throughout, and fully deterministic (no clocks, no ambient randomness; the demos use fixed-seed generators). This keeps it portable from desktop to embedded targets, and keeps every experiment repeatable — a prerequisite for the controlled-environment, learning-oriented use the project is intended for.

2 System overview

Module	Header	Responsibility
<code>mat</code>	<code>mv/mat.h</code>	small dense linear algebra, Jacobi SVD, null vectors
<code>cam</code>	<code>mv/cam.h</code>	camera model, projection, rays, distortion
<code>target</code>	<code>mv/target.h</code>	printed checkerboard model + printable rendering
<code>graycode</code>	<code>mv/graycode.h</code>	display-based active target (Gray-code patterns)
<code>calib</code>	<code>mv/calib.h</code>	homographies; Zhang calibration; radial estimate
<code>epipolar</code>	<code>mv/epipolar.h</code>	$\mathbf{F}$ from calibration; normalized 8-point; $\mathbf{E}$
<code>triangulate</code>	<code>mv/triangulate.h</code>	N -view DLT triangulation, reprojection RMS
<code>rectify</code>	<code>mv/rectify.h</code>	Fusiello rectification, disparity→depth
<code>stereo</code>	<code>mv/stereo.h</code>	dense block matching (SAD, sub-pixel)
<code>img</code>	<code>mv/img.h</code>	PGM (P5) image I/O
<code>cloud</code>	<code>mv/cloud.h</code>	point cloud container, PLY export

Table 1: Module layout. `mv/mv.h` is the umbrella header.

Processing has a one-time phase and a per-frame phase:

$\underbrace{\text{target views} \rightarrow \text{calibrate}}_{\text{once, target/graycode/calib}}$	$\underbrace{2 \text{ frames} \rightarrow \text{undistort} \rightarrow \text{rectify} \rightarrow \text{match} \rightarrow \text{depth} \rightarrow \text{cloud}}_{\text{per frame, cam/rectify/stereo/triangulate/cloud}}$
--	---

Sparse operation (feature correspondences instead of dense matching) uses the same chain with **epipolar** providing the correspondence-validation gate. Because the rig is static, temporal accumulation over video frames is a simple union of per-frame clouds in the fixed world frame; moving objects appear as temporally inconsistent points and can be segmented by comparing frames — the entry point for scene *analysis* beyond reconstruction.

3 Mathematical toolkit

Four ideas carry the whole system; everything later is an application.

3.1 Homogeneous coordinates

A 2D point (u, v) is represented as any triple (wu, wv, w) , $w \neq 0$; a 3D point (X, Y, Z) as $(X, Y, Z, 1)$ up to scale. The payoff: projection, homographies and rigid motions all become *linear* maps, and points at infinity ($w = 0$) need no special case. We write $\simeq$ for equality up to a nonzero scale.

Example 1 (dehomogenization). $(120, 140, 2) \simeq (60, 70, 1)$: both name the pixel $(60, 70)$. Conversely the direction “straight down the u axis” is $(1, 0, 0)$, which no finite pixel represents.

3.2 Rotations

A rotation matrix $\mathbf{R} \in SO(3)$ satisfies $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$; its rows are the axes of the new frame expressed in the old. Consequently $\mathbf{R}^{-1} = \mathbf{R}^\top$ — inverting a rigid transform never requires a matrix inversion.

Example 2 (rotation about y). $\mathbf{R}_y(90^\circ) = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$ sends the world z axis $(0, 0, 1)$ to $(-1, 0, 0)$: a camera yawed 90° sees the world’s $-x$ direction ahead. Check $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ by inspection: the rows are orthonormal.

3.3 The cross-product matrix

For $\mathbf{v} = (v_1, v_2, v_3)$,

$$[\mathbf{v}]_\times = \begin{pmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{pmatrix}, \quad [\mathbf{v}]_\times \mathbf{w} = \mathbf{v} \times \mathbf{w}. \quad (1)$$

It is rank 2 with null vector $\mathbf{v}$ itself — the algebraic reason epipoles exist (section 6).

Example 3 (skew matrix). $\mathbf{v} = (1, 0, 0)$: $[\mathbf{v}]_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$, and $[\mathbf{v}]_\times (0, 1, 0)^\top = (0, 0, 1)^\top = \mathbf{v} \times \mathbf{w}$.

3.4 Homogeneous least squares and the SVD

Every estimation problem in this system reduces to: given $\mathbf{A}$ ($m \times n$), find $\mathbf{x} \neq 0$ minimizing $\|\mathbf{Ax}\|$ subject to $\|\mathbf{x}\| = 1$. The minimizer is the right singular vector of the smallest singular value of $\mathbf{A} = \mathbf{U} \text{diag}(\sigma_1 \geq \dots \geq \sigma_n) \mathbf{V}^\top$ [GV13]. The library computes this with a one-sided Jacobi SVD (section 12) via `mv_nullvec`.

Example 4 (null vector by hand). $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ has rank 1 (row 2 = 2 × row 1), so the smallest singular value is exactly 0 and the null vector solves $x_1 + 2x_2 = 0$: $\mathbf{x} = \frac{1}{\sqrt{5}}(2, -1)$. Check: $\mathbf{Ax} = \mathbf{0}$.

4 Camera model

A camera is a pinhole with intrinsic matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (2)$$

rotation $\mathbf{R}$ and translation $\mathbf{t}$ mapping world points into the camera frame, $\mathbf{X}_c = \mathbf{RX} + \mathbf{t}$; the camera center is $\mathbf{C} = -\mathbf{R}^\top \mathbf{t}$. A world point projects to the homogeneous pixel

$$\mathbf{x} \simeq \mathbf{P} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix}, \quad \mathbf{P} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \quad (3)$$

[HZ04]. Real lenses deviate from (3); on normalized coordinates $(x, y) = (X_c/Z_c, Y_c/Z_c)$ with $r^2 = x^2 + y^2$ the Brown model gives the distorted

$$\begin{aligned} x_d &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \\ y_d &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \end{aligned} \quad (4)$$

and the pixel is $\mathbf{K}(x_d, y_d, 1)^\top$. Back-projection inverts (4) by fixed-point iteration and returns the ray $\mathbf{C} + s\mathbf{d}$, $s > 0$.

Example 5 (projection). Let $f_x=f_y=100$, $c_x=c_y=50$, $s=0$, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = \mathbf{0}$, no distortion, and $\mathbf{X} = (1, 2, 10)$. Normalized coordinates: $(x, y) = (1/10, 2/10) = (0.1, 0.2)$. Pixel: $u = 100 \cdot 0.1 + 50 = 60$, $v = 100 \cdot 0.2 + 50 = 70$ — the point lands at $(60, 70)$, cf. [Example 1](#).

Example 6 (camera center). Let $\mathbf{R} = \mathbf{R}_y(90^\circ)$ from [Example 2](#) and $\mathbf{t} = (3, 0, 0)$. Then $\mathbf{R}^\top \mathbf{t} = (0, 0, -3)$ and $\mathbf{C} = -\mathbf{R}^\top \mathbf{t} = (0, 0, 3)$: the camera sits three units down the world z axis. Check forward: $\mathbf{RC} + \mathbf{t} = (0 \cdot 0 + 0 \cdot 0 - 1 \cdot 3, 0, 0 \cdot 3) + (3, 0, 0) = (-3, 0, 0) + (3, 0, 0) = \mathbf{0}$ — the center maps to the camera origin, as it must.

5 Calibration from planar targets

Calibration is part of the system, not an external input. Both supported targets are *planes with known metric structure*: a printed checkerboard page ([subsection 5.4](#)) and a controlled computer display ([subsection 5.5](#)). The mathematics is the same for both: several views of a known plane determine the intrinsics, the distortion, and each view's pose, by Zhang's method [Zha00, SM99].

5.1 The homography of a plane

Put the target plane at $Z = 0$ in its own frame. Then (3) collapses: writing $\mathbf{R} = [\mathbf{r}_1 \mathbf{r}_2 \mathbf{r}_3]$ (columns),

$$\mathbf{x} \simeq \mathbf{K} [\mathbf{r}_1 \mathbf{r}_2 \mathbf{t}] (X, Y, 1)^\top = \mathbf{H} (X, Y, 1)^\top, \quad (5)$$

a 3×3 *homography* with 8 degrees of freedom. Each point correspondence gives two linear equations in the entries of $\mathbf{H}$, so $n \geq 4$ points determine it as a homogeneous least-squares problem (Example 4 at scale $2n \times 9$), solved by normalized DLT (`mv_homography_dlt`; normalization as in section 12).

Example 7 (plane homography). Frontal camera: $\mathbf{K}$ as in Example 5, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = (0, 0, 2)$. Then $[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}] = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix}$ and

$$\mathbf{H} = \mathbf{K} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 100 & 0 & 100 \\ 0 & 100 & 100 \\ 0 & 0 & 2 \end{pmatrix} \simeq \begin{pmatrix} 50 & 0 & 50 \\ 0 & 50 & 50 \\ 0 & 0 & 1 \end{pmatrix}.$$

The plane point $(X, Y) = (0.1, 0.2)$ maps to $(50 \cdot 0.1 + 50, 50 \cdot 0.2 + 50) = (55, 60)$. Check via Example 5: the 3D point is $(0.1, 0.2, 2)$, normalized $(0.05, 0.1)$, pixel $(100 \cdot 0.05 + 50, 100 \cdot 0.1 + 50) = (55, 60)$.

5.2 Zhang's constraints and the closed-form intrinsics

Write $\mathbf{H} = [\mathbf{h}_1 \ \mathbf{h}_2 \ \mathbf{h}_3]$ (columns). From (5), $\mathbf{h}_1 \simeq \mathbf{K}\mathbf{r}_1$ and $\mathbf{h}_2 \simeq \mathbf{K}\mathbf{r}_2$ with the *same* scale. Since $\mathbf{r}_1, \mathbf{r}_2$ are orthonormal,

$$\mathbf{h}_1^\top \mathbf{B} \mathbf{h}_2 = 0, \quad \mathbf{h}_1^\top \mathbf{B} \mathbf{h}_1 = \mathbf{h}_2^\top \mathbf{B} \mathbf{h}_2, \quad \mathbf{B} := \mathbf{K}^{-\top} \mathbf{K}^{-1}, \quad (6)$$

i.e. each view of the plane contributes two linear equations in the six independent entries of the symmetric matrix $\mathbf{B}$ (the *image of the absolute conic*). Three views generically determine $\mathbf{B}$ up to scale; two suffice if zero skew ($s=0 \Leftrightarrow B_{12}=0$) is imposed. The solution is again a null vector (Example 4); $\mathbf{K}$ then follows from $\mathbf{B}$ in closed form [Zha00], and the pose of each view is recovered as

$$\lambda = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_1\|}, \quad \mathbf{r}_1 = \lambda \mathbf{K}^{-1}\mathbf{h}_1, \quad \mathbf{r}_2 = \lambda \mathbf{K}^{-1}\mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \lambda \mathbf{K}^{-1}\mathbf{h}_3, \quad (7)$$

with $[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]$ snapped to the nearest true rotation via SVD (noise makes it only approximately orthonormal).

Example 8 (pose from a homography, by hand). Take the (unscaled) $\mathbf{H}$ of Example 7 and the known $\mathbf{K}$. Then

$$\mathbf{K}^{-1} = \begin{pmatrix} 0.01 & 0 & -0.5 \\ 0 & 0.01 & -0.5 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_3 = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}.$$

Both constraint equations of (6) hold: $\mathbf{r}_1 \cdot \mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$. The scale is $\lambda = 1/\|(1, 0, 0)\| = 1$, so (7) gives $\mathbf{r}_1 = (1, 0, 0)$, $\mathbf{r}_2 = (0, 1, 0)$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = (0, 0, 1)$, $\mathbf{t} = (0, 0, 2)$ — exactly the frontal pose two metres from the target that we started from.

5.3 Radial distortion and alternation

With $\mathbf{K}$ and poses in hand, the ideal (undistorted) projection (u, v) of every target point is known, and (4) predicts the observed (u_d, v_d) linearly in (k_1, k_2) :

$$u_d - u = (u - c_x)(k_1 r^2 + k_2 r^4), \quad v_d - v = (v - c_y)(k_1 r^2 + k_2 r^4), \quad (8)$$

a least-squares problem with two unknowns. Since the homographies were fit to *distorted* points, the library alternates: fit geometry, estimate (k_1, k_2) , undistort the observations, refit — a few rounds converge for moderate distortion [Zha00].

An identifiability caveat worth internalizing: over the small radius range a letter-page target covers, r^2 and r^4 are nearly proportional, so k_1 and k_2 are individually ill-determined even when their combined *curve* $k_1 r^2 + k_2 r^4$ — the thing that actually corrects pixels — is fit to well below noise (measured in [section 13](#)). Wide targets, or the full-field display target of [subsection 5.5](#), are the cure when the individual coefficients matter.

5.4 The printed target

The standard passive target is a checkerboard printed on a letter/A4 page: 8×10 squares of nominally 24 mm (192×240 mm), i.e. 7×9 inner corners (`mv/target.h`; `mv_target_render_pgm` emits the printable image — at 10 px/mm a 24 mm square is 240 px). Printers rescale, so the procedure is *print, then measure one square* and pass the measured size; only that measurement makes the calibration metric. Corner *positions* in the image are currently supplied by the caller (an external detector or manual marking); a saddle-point corner detector is on the roadmap ([section 14](#)). Ten to twenty views with strong tilt diversity (± 30 – 40°) condition the focal length well — near-frontal views leave it weakly observable, a property visible in the experiment of [section 13](#).

5.5 The display as an active target

A computer display we control is a better target than paper wherever one is available: its pixel grid is a plane of exactly known geometry (pixel pitch from the panel datasheet, or measured once), and because we control what it shows *when*, correspondence needs no detector at all. The display plays, synchronized with capture, a sequence of binary stripe patterns that encode each display pixel’s coordinate in *Gray code* [ISM84, Gen11]: bit b of the code is shown as stripes, followed by its inverse, for $b = 0..[\log_2 W]-1$, then the same for the y axis. A camera pixel observing the display reads its bit as “brighter in the pattern or in the inverse?” — a per-pixel comparison immune to illumination, display brightness and gamma — and after all frames has decoded *which display pixel it sees* (`mv/graycode.h`). This yields thousands of dense, uniformly spread correspondences per view; the same pipeline ([subsection 5.1](#)–[subsection 5.3](#)) consumes them, with plane coordinates = display pixel $\times$ pitch. Gray code rather than plain binary because adjacent stripes differ in exactly one bit: a threshold error at a stripe boundary displaces the decoded coordinate by at most one pixel instead of, e.g., 2^b .

Example 9 (Gray code by hand). Encode $v = 6$: binary 110; Gray = $110 \oplus 011 = 101$. Decode $g = 101$ by prefix-XOR: $b_2 = 1$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 1 \oplus 0 \oplus 1 = 0$ — binary 110 = 6. Now flip the last Gray bit (a boundary misread): $g' = 100 \rightarrow$ binary 111 = 7, an error of exactly one position. In plain binary, flipping the top bit of 110 gives 010 = 2, an error of four.

5.6 Design: the rotating display

For an N -camera rig ([section 10](#)) the planned calibration fixture is the display of [subsection 5.5](#) mounted on a plate rotating at constant angular velocity ω , so that the screen sweeps past every camera each revolution. This subsection records the design; implementation is a roadmap item ([section 14](#)).

What the rotation provides. Three things, all substantial. (i) *Tilt diversity per camera*: as the screen sweeps past, every frame shows the known plane at a different orientation — exactly the varied-tilt view set Zhang’s method needs, whose absence measurably degrades focal recovery (section 13). (ii) *One shared orbit*: every camera observes the same one-parameter family of poses — screen pose = mount offset composed with rotation about a fixed axis at constant ω . Fitting that orbit (axis, ω , phase, mount; ~ 10 parameters) jointly across cameras links *all* of them into one world frame even if no two ever see the screen simultaneously: the orbit itself is the common reference object, replacing pairwise shared views. (iii) *The display is the clock*: the screen broadcasts its frame index, so every captured image self-reports display time and cameras synchronize to the rotation model without a hardware trigger.

What the rotation forbids. The static Gray-code stack of subsection 5.5 requires one camera pixel to observe one display point across ~ 20 frames — impossible on a moving target. The pattern must therefore be *single-shot self-identifying*: every frame decodable alone. The design is layered:

- a checkerboard-type *saddle-point grid* as the metric layer — saddle locations are unbiased under perspective and under (moderate, symmetric) blur, unlike dot centroids, which both foreshortening and blur displace;
- *identity coding* embedded in the grid (marker bits in alternate squares, or an M-array whose every $k \times k$ window is unique), so partial and oblique views self-locate — most frames of a sweep are partial;
- a *frame-counter strip* along the border, Gray-coded *in time* (successive indices differ in one bit, cf. Example 9), providing the timestamp for the orbit model;
- *multi-scale structure* (a coarse grid nested with a fine one), since one display serves cameras at different distances and obliquities; strokes must span ≥ 3 –4 pixels in the worst camera at the most oblique usable pose;
- *binary black/white only*: LCD gamma and contrast shift off-axis, so gray-level codes degrade with viewing angle while binary features with per-frame identity do not.

Across frames, only the identity layer changes (counter strip; optionally alternating coarse/fine emphasis) — pose variation is already supplied by the rotation. At low ω the screen moves < 1 pixel between consecutive frames, so pattern-plus-inverse in consecutive frames remains usable and restores the per-pixel contrast robustness of subsection 5.5.

Example 10 (motion-blur budget). Image-space blur during exposure t_e is $\omega r f t_e / Z$ pixels, with r the lever arm from axis to screen edge. For the experimental rig’s numbers ($f = 800$ px, $Z = 4.7$ m) with $r = 0.25$ m and $\omega = 6$ rpm = 0.63 rad/s: the screen edge moves $0.63 \cdot 0.25 \cdot 800 / 4.7 \approx 27$ px/s, so a 5 ms exposure blurs 0.13 px — comfortably sub-pixel, and between frames at 30 fps the screen shifts 0.9 px, validating the consecutive-inverse-frame option. The formula is the design knob linking ω , exposure, and screen brightness.

One mechanical trap: cant the screen. A screen standing parallel to the rotation axis presents orientations that all differ by rotation about a *single* axis — a weak/singular family for plane-based calibration [SM99], the same disease as the measured near-frontal weakness.

Mounting the screen tilted back 20–30° makes its normal sweep a cone instead of a circle, exercising two rotation axes; this one bracket angle is the most important mechanical detail of the fixture.

Pipeline. Gate frames by obliquity (roughly 15–55° off-normal: frontal frames are uninformative, grazing frames unmatchable); per-frame homographies from the self-identifying grid; per-camera Zhang intrinsics from the gated set; joint orbit fit for all extrinsics + axis + ω + phase — the natural first customer for the planned Levenberg–Marquardt stage.

6 Two-view epipolar geometry

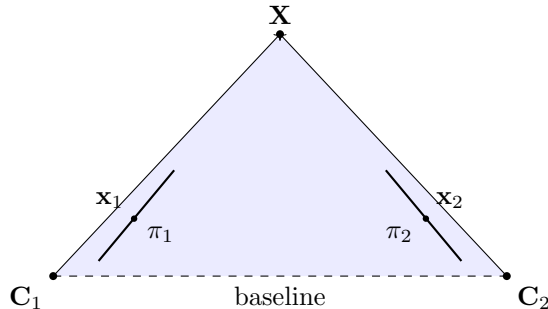


Figure 1: Epipolar geometry: the camera centers C_1 , C_2 and the scene point X span the epipolar plane (shaded); its intersections with the image planes π_1 , π_2 are the corresponding epipolar lines on which x_1 and x_2 must lie.

Corresponding pixels of the two views are linked by the fundamental matrix F through the epipolar constraint

$$x_2^T F x_1 = 0, \quad (9)$$

with F of rank 2; the line $\ell_2 = F x_1$ is the locus in image 2 of possible matches of x_1 (Figure 1). For *calibrated* views the essential matrix $E = K_2^T F K_1 = [t]_{\times} R$ carries the same constraint in normalized coordinates [LH81]; note the reappearance of (1).

Because our rig is calibrated, F is available in closed form,

$$F = [e_2]_{\times} P_2 P_1^+, \quad e_2 = P_2 \begin{pmatrix} C_1 \\ 1 \end{pmatrix}, \quad (10)$$

where P_1^+ is the pseudo-inverse of P_1 [HZ04]. The system computes (10) once at start-up and uses it for *correspondence gating*: any candidate match whose symmetric epipolar distance exceeds a threshold is rejected before triangulation.

Independently, F can be *estimated* from $n \geq 8$ correspondences by the normalized 8-point algorithm [Har97]: each pair turns (9) into one linear equation in the nine entries of F — again the homogeneous least-squares pattern of Example 4 — followed by rank-2 enforcement via SVD. In this system the estimator serves two purposes: (i) *health monitoring* — if the estimate from live correspondences drifts from the calibrated (10), the rig has been bumped or the calibration is stale; (ii) bootstrapping geometry for *uncalibrated* auxiliary photos (section 11). On contaminated correspondence sets the estimator is wrapped in a RANSAC loop [FB81] (planned with the feature layer).

Example 11 (essential matrix of a rectified rig). Two identical cameras, the second displaced one unit along x : $\mathbf{R} = \mathbf{I}$, $\mathbf{C}_2 = (1, 0, 0)$, so $\mathbf{t} = -\mathbf{R}\mathbf{C}_2 = (-1, 0, 0)$. By [Example 3](#),

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \quad \mathbf{x}_2^{\top} \mathbf{E} \mathbf{x}_1 = (x_2, y_2, 1) \begin{pmatrix} 0 \\ 1 \\ -y_1 \end{pmatrix} = y_2 - y_1.$$

The epipolar constraint degenerates to $y_2 = y_1$: corresponding points lie on the *same image row*. This is precisely the situation rectification ([section 8](#)) manufactures for arbitrary rigs.

7 Triangulation

Given observations $\mathbf{x}^{(v)} = (u_v, w_v)$ of the same point in $N \geq 2$ views with projection matrices $\mathbf{P}^{(v)}$, each view contributes two linear equations obtained by eliminating the projective scale from (3):

$$\begin{pmatrix} u_v \mathbf{P}_{3,:}^{(v)} - \mathbf{P}_{1,:}^{(v)} \\ w_v \mathbf{P}_{3,:}^{(v)} - \mathbf{P}_{2,:}^{(v)} \end{pmatrix} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix} = \mathbf{0}, \quad (11)$$

and stacking all views gives a $2N \times 4$ homogeneous system solved, once more, as in [Example 4](#) — the DLT triangulation [[HZ04](#)]. The implementation accepts any $N \geq 2$, so still photographs registered into the rig frame ([section 11](#)) simply extend the same call; what additional views and additional pixels buy, quantitatively, is analyzed in [section 10](#). DLT minimizes an algebraic residual; the reprojection-optimal two-view method of Hartley–Sturm [[HS97](#)] is a planned refinement, worthwhile mainly for wide-baseline auxiliary views. The library reports the RMS reprojection error of every triangulated point — the quantity an eventual bundle adjustment would minimize [[TMHF00](#)].

Example 12 (triangulation by similar triangles). Rig of [Example 11](#) with $\mathbf{K}$ of [Example 5](#) ($f = 100$, $c = 50$) and baseline $B = 0.2$: camera 2 at $(0.2, 0, 0)$. Scene point $\mathbf{X} = (0.1, 0, 2)$. View 1: $(x, y) = (0.05, 0)$, pixel $u_1 = 100 \cdot 0.05 + 50 = 55$. View 2: camera coordinates $(0.1 - 0.2, 0, 2) = (-0.1, 0, 2)$, so $u_2 = 100 \cdot (-0.05) + 50 = 45$. Disparity $d = u_1 - u_2 = 10$, and (12) gives $Z = fB/d = 100 \cdot 0.2/10 = 2$ ✓. The lateral coordinate follows from view 1: $X = Z(u_1 - c_x)/f = 2 \cdot 5/100 = 0.1$ ✓ — the full point recovered by hand.

8 Rectification and dense stereo

For dense per-pixel depth the pair is first *rectified*: both cameras are rotated (synthetically, about their centers) onto a common orientation whose x axis is the baseline, making the geometry of [Example 11](#) true by construction — epipolar lines become identical image rows and correspondence search collapses to 1D. We use the compact algorithm of Fusiello, Trucco and Verri [[FTV00](#)]: the new rotation has rows $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ with $\mathbf{x}$ the unit baseline, $\mathbf{y} = \mathbf{k} \times \mathbf{x}$ ($\mathbf{k}$ the old optical axis of camera 1) and $\mathbf{z} = \mathbf{x} \times \mathbf{y}$; both views share the averaged intrinsics with zero skew, and the pixel maps are the homographies $\mathbf{H}_i = \mathbf{K}_{\text{new}} \mathbf{R}_{\text{new}} \mathbf{R}_i^{\top} \mathbf{K}_i^{-1}$ (homographies again — cf. (5), here induced by pure rotation instead of a scene plane). Since the rig is stationary, $\mathbf{H}_1, \mathbf{H}_2$ are computed once and each video frame is warped by table lookup.

On the rectified pair, dense correspondence is found by block matching: for each left pixel the sum of absolute differences (SAD) over a $(2w+1)^2$ window is minimized along the corresponding

right row over a disparity range, followed by sub-pixel refinement by parabolic interpolation of the cost around the minimum. This is the classic local baseline method in the taxonomy of Scharstein and Szeliski [SS02] — chosen deliberately: it is simple, portable, trivially parallel, and its failure modes (textureless regions, occlusions, repetitive patterns) are well understood, giving a controlled baseline against which future matchers (rank/census transforms, semi-global aggregation) can be measured within the same harness.

Matched disparity d converts to metric depth by

$$Z = \frac{f B}{d}, \quad (12)$$

with f the shared rectified focal length (pixels) and B the baseline. Differentiating (12) gives the expected depth uncertainty

$$\sigma_Z = \frac{Z^2}{f B} \sigma_d \quad (13)$$

for disparity noise σ_d — the quadratic growth with Z is the fundamental accuracy law of the rig.

Example 13 (depth error budget). The experimental rig of [section 13](#): $f = 800$ px, $B = 0.5$ m, mean depth $Z = 4.7$ m, per-coordinate noise $\sigma = 0.3$ px so $\sigma_d = \sqrt{2} \sigma \approx 0.42$ px. Then $\sigma_Z = 4.7^2 \cdot 0.42 / (800 \cdot 0.5) \approx 0.023$ m: about 2.3 cm of depth uncertainty — compare the measured 2.65 cm RMS 3D error in [Table 2](#).

9 From depth to models

Every depth estimate back-projects to a 3D point in the fixed world frame; points are accumulated in a growable cloud with optional per-point color and exported as ASCII PLY, directly loadable in MeshLab, CloudCompare or Blender for inspection. Over a video sequence the static-rig assumption makes accumulation a union in a common frame; per-point temporal statistics (persistence, variance) separate static structure from moving objects. The planned meshing path is classical: volumetric range-image fusion [CL96] or Poisson surface reconstruction [KBH06] over the accumulated cloud; both consume exactly the oriented-point data the pipeline produces, so they attach cleanly as later modules.

10 Scaling the rig: more cameras and more pixels

The two-camera rig is the minimum that triangulates. This section works out what changes when the rig grows — N cameras around the same volume, or higher (possibly unequal) resolutions — so that hardware decisions can be made from the error laws instead of intuition. The library is already shaped for this: `mv_triangulate` accepts any $N \geq 2$, and every camera carries its own $\mathbf{K}$, so heterogeneous rigs need no new code paths in the sparse pipeline.

10.1 N cameras: accuracy

Adding cameras improves triangulation through two distinct mechanisms.

Geometry (the dominant effect). The depth law (13) is governed by the baseline: rays from nearby centers intersect at grazing angles, and the intersection point slides far along the rays for small image perturbations. With cameras spread out, the *widest* usable pair sets the leading term: for N collinear cameras spaced B apart, the widest baseline is $(N-1)B$ and

$$\sigma_Z^{\text{widest}} = \frac{Z^2}{f(N-1)B} \sigma_d, \quad (14)$$

an $(N-1)$ -fold improvement before any averaging. The catch is correspondence: wide pairs see the surface from very different angles, so matching is harder — which is exactly why multi-baseline stereo [OK93] matches *short* pairs (easy, unambiguous) and accumulates their evidence in a common inverse-depth parameterization, getting wide-pair accuracy at short-pair matching difficulty. Repetitive texture that fools one pair (a periodic fence aliases at one baseline) is disambiguated by any other baseline whose period does not divide it — the match must be consistent across *all* pairs simultaneously.

Redundancy. With visibility and geometry held fixed, N views of the same point give the usual statistical $\sqrt{N}$: the DLT stack (11) grows to $2N \times 4$ rows and the estimate averages the per-view noise. This is the smaller effect — doubling N buys $\sqrt{2} \approx 1.4\times$; doubling the *spread* buys $2\times$.

Example 14 (four collinear cameras). Extend the experimental rig of Example 13 ($f = 800$ px, $Z = 4.7$ m, $\sigma_d = 0.42$ px) from two cameras 0.5 m apart to four cameras at 0, 0.5, 1.0, 1.5 m. The widest pair has $B = 1.5$ m, so by (14) $\sigma_Z = 4.7^2 \cdot 0.42 / (800 \cdot 1.5) \approx 7.7$ mm — three times better than the 23 mm of the original pair, from geometry alone; fusing the remaining pairs tightens it further. The price: the widest pair’s viewing directions differ by $2 \arctan(0.75/4.7) \approx 18^\circ$, and matching windows must survive that foreshortening — the Okutomi–Kanade argument for accumulating the short pairs instead.

10.2 N cameras: occlusion and topology

A point enters the model only if at least two cameras see it. With two cameras, anything hidden from either one is lost; the reconstructed surface is only the band that is simultaneously front-facing and unobstructed for both — for objects with complex topology (handles, tunnels, deep concavities) that band can be a small fraction of the surface, and the holes it leaves are the familiar occlusion artifacts.

A useful back-of-envelope model: if a surface point is occluded in any single view independently with probability q , then

$$P(\text{reconstructible}) = P(\geq 2 \text{ of } N \text{ see it}) = 1 - q^N - N q^{N-1}(1 - q). \quad (15)$$

Independence is optimistic for clustered cameras (they get occluded together) and pessimistic for well-spread ones — spreading the cameras is what makes the assumption approachable.

Example 15 (occlusion coverage). Take $q = 0.3$ (a fairly convoluted scene). By (15): $N = 2$: $(1 - q)^2 = 0.49$ — half the surface. $N = 4$: $1 - 0.3^4 - 4 \cdot 0.3^3 \cdot 0.7 = 1 - 0.0081 - 0.0756 \approx 0.92$. $N = 6$: $1 - 0.3^6 - 6 \cdot 0.3^5 \cdot 0.7 \approx 0.99$. Going from two to four cameras nearly doubles the reconstructible surface; four to six mops up the remainder. The marginal camera always buys less — but for high-genus objects the last percent *is* the topology.

Topology sharpens the point. Silhouette-based reasoning can never recover concavities: the best any number of silhouettes gives is the visual hull [Lau94], which fills in bowls and blocks handles. Capturing the true genus of an object — the hole of a handle, the tunnel through an arch — requires cameras whose *sightlines pass through the hole*, seeing background where the hull would put material; photo-consistency frameworks (space carving [KS00], patch-based multiview stereo [FP10]) then remove exactly the voxels/patches contradicted by those views. So for complex topology, added cameras should be *spread to thread the object’s openings and look into its concavities*, not packed for baseline; the standard benchmark evidence [SCD⁺06] is that view coverage, more than per-view quality, is what separates complete reconstructions from holed ones. In this system’s terms: the two rig cameras provide the calibrated metric backbone, and subsection 11.1’s registered auxiliary photos are the cheap way to add exactly such coverage views.

Two practical costs scale with N : (i) calibration and synchronization — each camera needs its own section 5 session against the same target, all cameras a common trigger; (ii) computation — pairwise geometry (fundamental matrices, rectifications) grows as $N(N-1)/2$, though in practice a hub-and-spoke set against one reference camera suffices for gating.

10.3 Resolution

For a fixed field of view, the focal length *in pixels* scales with the sensor’s linear resolution: doubling from 640×480 to 1280×960 doubles f from 800 to 1600 px. Two consequences follow directly.

Depth precision. Matching accuracy σ_d is an approximately constant *fraction of a pixel* (our block matcher’s parabolic refinement sits around 0.1–0.3 px on textured surfaces), so in (13) the denominator’s f doubles while σ_d stays put: σ_Z halves. Depth precision is linear in resolution — until lens blur, diffraction, or per-pixel photon noise (smaller pixels collect less light) starts inflating σ_d and returns diminish.

Lateral detail. The pixel footprint on the scene (ground sampling distance) is Z/f metres per pixel. Surface detail must span at least ~ 2 pixels to be matchable, so the smallest reconstructible geometric feature is about $2Z/f$ — this, not depth noise, is what limits how much of a complex object’s fine structure (thin ribs, small holes) survives into the model. In the fidelity sense of subsection 10.2: resolution sets the smallest feature you can capture, view coverage sets whether you capture it at all.

Example 16 (resolution budget). Experimental rig, $Z = 4.7$ m: footprint $Z/f = 4.7/800 = 5.9$ mm/px, smallest matchable feature ≈ 12 mm, $\sigma_Z \approx 23$ mm (Example 13). Doubling both cameras’ resolution: footprint 2.9 mm/px, feature floor ≈ 6 mm, $\sigma_Z \approx 12$ mm. Everything improves by the same factor of two.

10.4 Unequal resolutions

Nothing in the model requires the cameras to match: each carries its own $\mathbf{K}$ (and distortion), the fundamental matrix (10) exists between any pair, and triangulation consumes pixels through each camera’s own $\mathbf{P}$. What changes is the error budget. Each camera contributes *angular* noise

σ/f_i (radians); ray-intersection variance adds them, so a pair’s effective noise is

$$\sigma_\theta = \sqrt{(\sigma_1/f_1)^2 + (\sigma_2/f_2)^2} : \quad (16)$$

the *coarser camera dominates*, and money spent on one camera is mostly wasted unless the other keeps up.

Example 17 (upgrading one camera vs both). Rig of [Example 13](#), $\sigma = 0.3$ px per camera. Both at $f = 800$: $\sigma_\theta = \sqrt{2}(0.3/800)$. Upgrade *one* camera to $f = 1600$: $\sigma_\theta = (0.3/800)\sqrt{1 + \frac{1}{4}}$, an improvement of only $\sqrt{1.25/2} \approx 0.79$ — depth error drops from 23 mm to 18 mm. Upgrade *both*: factor 0.5, 12 mm ([Example 16](#)). Half the hardware upgrade buys about a fifth of the benefit.

Two implementation notes for mixed rigs. First, rectification ([section 8](#)) forces a *shared* $\mathbf{K}_{\text{new}}$; our implementation averages the two intrinsic matrices, which for very unequal cameras oversamples the coarse image (interpolated pixels carry no new information) — resampling both to the coarser camera’s grid keeps the cost of dense matching honest, at the price of discarding the fine camera’s lateral detail. Second, the DLT ([11](#)) as implemented weights all rows equally; with strongly unequal cameras the rows of the finer camera deserve proportionally more weight ($\propto f_i/\sigma_i$ after each view’s normalization), a small planned refinement alongside the Hartley–Sturm step ([section 14](#)).

11 Planned extensions

11.1 Auxiliary photo collections

Still photographs of the same volume — arbitrary viewpoints, possibly a different camera — densify coverage and fill occlusions. The integration path follows the structure-from-motion recipe of Photo Tourism [[SSS06](#)]: detect scale-invariant features [[Low04](#)] in the photos *and* in the two calibrated video frames; match with a RANSAC gate [[FB81](#)]; register each photo to the rig’s world frame by resection from matches against already-triangulated points; then re-triangulate all tracks with the registered views via the existing N -view DLT ([section 7](#)); finally refine everything with bundle adjustment [[TMHF00](#)]. The two calibrated cameras anchor the metric scale — the classic scale ambiguity of pure SfM does not arise. Dense multiview stereo over the registered set (e.g. patch-based methods [[FP10](#)]) is a further optional stage.

11.2 Range-finder fusion (optional, kept open)

The system is designed to work without a range finder; this subsection records what a single-beam laser ranger would add if one arrives later, so the option stays open at zero present cost. Two integration modes: (i) if the range finder is rigidly mounted and extrinsically calibrated to the rig, each range sample is simply another world-frame point with small σ_Z , fused into the cloud with inverse-variance weights; (ii) if it produces an independent scan, the scan is registered to the vision cloud by closed-form absolute orientation [[Hor87](#)] initialized correspondence-free and refined with ICP [[BM92](#)].

At calibration time (with the rotating display of [subsection 5.6](#)). Four distinct improvements. (i) *Independent absolute scale*: the vision-only metric chain hangs entirely on the display’s pixel pitch; a ~ 2 mm-accurate range at 4.7 m is a 0.04% scale reference — a second

chain that turns a silent global scale bias into a detectable discrepancy. (ii) *Orbit metrology*: aimed so the rotating screen sweeps through the beam, the periodic range trace $d(t)$ yields ω , phase, orbit radius and axis distance at millimetre level, independently of every camera, and ties laser and camera timestamps to the same physical event. (iii) *The focal–depth weak direction* (subsection 5.4) is attacked head-on: an absolute distance to the target plane is exactly the constraint type planar homographies lack. (iv) *The plate calibrates the laser*: if the spot is visible to the cameras, the sweeping plate carries it through continuously varying depths; each frame gives (range, pixel₁, pixel₂), the cameras triangulate the spot, and the beam’s ray in the rig frame (~ 5 DOF) falls out of a least-squares line fit — no manual procedure at all.

At run time. Laser error is roughly constant while stereo error grows as Z^2 (13), so the laser anchors the far field and the absolute scale (sparse metric skeleton, dense stereo flesh); it provides depth in texture deserts where block matching has nothing to match, and used as a prior it narrows the disparity search there; and a persistent bias between laser and stereo depth at the same spot is an online health monitor for the metric chain, complementing the $\mathbf{F}$ -drift monitor of section 6.

Example 18 (laser–stereo crossover). With $\sigma_{\text{laser}} = 2$ mm constant and the rig’s $\sigma_Z = Z^2 \sigma_d / (fB)$: equality at $Z = \sqrt{0.002 \cdot 800 \cdot 0.5 / 0.42} \approx 1.4$ m. Nearer than that, stereo wins per point; beyond it the laser is better along its ray — tenfold better at 4.7 m (2 mm vs 23 mm) — while stereo retains the advantage of density. Inverse-variance fusion realizes both.

12 Numerical implementation notes

All decompositions reduce to one kernel: a one-sided (Hestenes) Jacobi SVD [GV13] for $m \geq n$ matrices, $n \leq 9$ in every present use (the design matrices of (5), (6), (11), and the 8-point system). Jacobi SVD was chosen over Golub–Kahan bidiagonalization for its simplicity (~ 100 lines, no workspace queries), unconditional convergence, and excellent relative accuracy on small matrices; its $O(mn^2)$ -per-sweep cost is irrelevant at these sizes. Homogeneous solutions are the right singular vector of the smallest singular value; rank-2 enforcement for $\mathbf{F}$ zeroes the smallest singular value and recomposes.

Conditioning is handled where it matters: design matrices built from raw pixel coordinates mix terms of order 1, u , and u^2 ($u \sim 10^2$ pixels), giving condition numbers around 10^8 ; Hartley normalization [Har97] (translate to centroid, scale to RMS radius $\sqrt{2}$) reduces this to $O(10)$ and is applied in both the 8-point and the homography estimators, with results denormalized afterwards. All fundamental matrices are canonicalized (unit Frobenius norm, largest-magnitude entry positive) so that analytic and estimated matrices are directly comparable — this is what makes the drift monitor of section 6 a plain matrix norm. Calibration runs Zhang’s closed form inside the small alternation loop of subsection 5.3; a full Levenberg–Marquardt refinement over all parameters is the planned upgrade (section 14).

The code is strict C99 (`-std=c99 -pedantic -Wall -Wextra` clean), depends only on `libm`, allocates only in unavoidable places (design matrices, images, clouds), and contains no hidden state except the demos’ seeded generators. Determinism is treated as a feature: identical inputs give bit-identical outputs across runs, so every regression is reproducible.

13 Basic results

Three validation layers accompany the library. `tests/test_mv.c` checks exact properties on noiseless data (30 assertions: SVD reconstruction and orthonormality, ray/projection consistency, (9) to 10^{-8} , exact homography and triangulation recovery, exact intrinsic/pose recovery by Zhang’s method, Gray-code encode/decode round-trip, row alignment after rectification); all pass. The two demo programs are controlled noisy experiments.

Two-camera reconstruction (`demo/synthetic.c`). Two cameras with $f = 800$ px, 640×480 images, baseline $B = 0.5$ m, the second camera yawed 6° inward, observe 250 points drawn uniformly in a 2×1.5 m window at depths 3.5–6 m; projections carry Gaussian noise $\sigma = 0.3$ px per coordinate (seed fixed).

Quantity	Value
points visible in both views	250/250
$\ \mathbf{F}_{\text{analytic}} - \mathbf{F}_{8\text{-point}}\ _F$ (unit-norm)	4.5×10^{-3}
mean symmetric epipolar distance (analytic $\mathbf{F}$)	0.345 px
mean symmetric epipolar distance (8-point $\mathbf{F}$)	0.341 px
triangulation RMS 3D error	0.0265 m
triangulation max 3D error	0.0876 m
RMS reprojection error	0.218 px
mean row misalignment $ v_1 - v_2 $ before rectification	0.764 px
mean row misalignment after rectification	0.0 px

Table 2: Reconstruction experiment, seed 42 (reproduce with `make demo`).

The numbers agree with theory: the epipolar residual ≈ 0.34 px matches the injected noise, and the RMS 3D error of 2.65 cm matches the 2.3 cm depth-noise floor computed by hand in [Example 13](#) (the measured value also contains the smaller lateral components). The implementation performs at the noise floor of the geometry rather than being implementation-limited. The reconstructed cloud is written to `out_cloud.ply`.

Calibration (`demo/calibrate.c`). A camera with ground-truth $f_x=800$, $f_y=805$, $(c_x, c_y)=(322, 238)$, zero skew and zero distortion photographs the letter-page target ([subsection 5.4](#); 7×9 corners, 24 mm squares) in six poses with tilts up to 35° at 0.45–0.65 m; corners carry $\sigma = 0.3$ px noise.

Quantity	Value
f_x error	−7.0 px (0.9%)
f_y error	−7.9 px (1.0%)
c_x, c_y error	−1.0, −1.8 px
mean rotation error over views	0.31°
mean translation error over views	5.2 mm
reprojection RMS	0.48 px

Table 3: Calibration experiment, seed 7, six views, $\sigma = 0.3$ px (reproduce with `make demo`).

Sub-1% intrinsics and millimetre-level poses from six noisy views of one printed page is the expected performance of Zhang’s method without nonlinear refinement; near-frontal pose sets degrade the focal estimate markedly (weak observability, [subsection 5.4](#)), which the experiment reproduces if the tilt angles are reduced. The fitted (k_1, k_2) absorb a little noise as a near-zero distortion *curve* while being individually large — the identifiability effect discussed in [subsection 5.3](#). A Levenberg–Marquardt refinement stage is the roadmap item that tightens all of these numbers.

14 Roadmap

In dependency order: (1) saddle-point checkerboard corner detection with sub-pixel refinement, closing the printed-target path end-to-end (the display path of [subsection 5.5](#) is already detector-free); (2) the rotating-display fixture of [subsection 5.6](#): single-shot self-identifying pattern generation (M-array + Gray-coded frame counter) and the joint orbit fit; (3) Levenberg–Marquardt refinement of calibration (and later bundle adjustment [[TMHF00](#)]); (4) feature detection/description and robust matching (enables live correspondences, RANSAC-gated by [section 6](#)); (5) image warping by the rectifying homographies with bilinear resampling, closing the dense video path end-to-end; (6) temporal cloud statistics for static/moving segmentation ([section 9](#)); (7) photo-collection registration ([subsection 11.1](#)); (8) Hartley–Sturm optimal triangulation; (9) meshing; and, if the hardware materializes, the optional range-finder integration of [subsection 11.2](#). Each step lands as a new module behind the same conventions ([Table 1](#)) with its own noiseless-exact tests and a synthetic experiment extending [section 13](#).

A Camera hardware and the quality of data

The capture hardware determines which error terms enter the pipeline before any algorithm runs. The organizing observation: cheap capture hardware mostly adds *biases* — rolling shear, synchronization parallax, compression structure, unmodelable distortion, focus creep — rather than noise. Noise we model and budget ([Example 13](#)); biases survive averaging, masquerade as geometry, and poison calibration silently. The specifications below are therefore ranked by which biases they admit.

A.1 Shutter: global vs rolling

A global-shutter sensor exposes all pixels over one common interval — each frame is a true snapshot, described by a single $\mathbf{P}$. A rolling-shutter sensor exposes and reads row by row: the bottom row of a frame is captured 10–30 ms after the top row. For a static scene the difference vanishes; for anything moving, each row samples a different instant and no single projection matrix describes the frame — the camera model of [section 4](#) itself breaks.

Example 19 (rolling-shutter shear on the rotating target). The rotating display of [subsection 5.6](#) moves at ≈ 27 px/s in the image. A rolling readout of 20 ms samples the top and bottom of the target 20 ms apart: a shear of $27 \cdot 0.02 \approx 0.5$ px across the corner grid — larger than the 0.3 px noise budget, and *systematic*: it does not average away over frames but feeds a consistently wrong homography into calibration.

A.2 Synchronization: hardware trigger vs free-running

A trigger line starts exposure on all cameras from one electrical pulse (coincidence to microseconds). Free-running cameras can only be matched by software timestamps — a few *milliseconds* of jitter and drift. Triangulation assumes both cameras saw the point at the same instant; a time offset Δt on a point moving at speed v creates a phantom disparity $f v \Delta t / Z$ that is mathematically indistinguishable from depth.

Example 20 (the price of software synchronization). An object crossing the experimental rig at 1 m/s with 5 ms software sync: phantom disparity = $800 \cdot (1 \cdot 0.005) / 4.7 \approx 0.85$ px, which the depth law (13) converts to ≈ 47 mm of depth error — exceeding the rig’s entire 23 mm error budget at that range. For dynamic scenes, synchronization error dominates every other term; a trigger wire outperforms any camera upgrade. The trigger also permits phase-locking exposure against mains flicker and locking capture to the display’s frame counter (subsection 5.6).

A.3 Interface: UVC vs machine-vision stacks

UVC (USB Video Class) is the “any webcam just works” protocol (v412 on Linux, no vendor SDK). Its convenience reflects its purpose — streaming video to humans: at higher resolutions frames usually arrive *MJPEG-compressed*, and JPEG’s 8×8 block artifacts are structured noise at exactly the spatial frequencies where sub-pixel corners and block matching live — a *correlated* contribution to σ_d that does not average like photon noise. UVC also offers only coarse loosely-specified exposure/gain controls, imprecise timestamps, and no trigger support in the class. Machine-vision interfaces (USB3 Vision/GenICam via vendor SDKs) deliver raw pixels, physical-unit controls, per-frame hardware timestamps and trigger I/O. Rule of thumb: UVC is acceptable exactly where rolling shutter is acceptable — static bring-up.

A.4 Optics: M12 vs C/CS mounts

M12 (“S-mount”) lenses are the small threaded board lenses of webcam-class hardware: cheap, fixed aperture, focused by rotating the barrel, no standardized flange distance. C-mount and CS-mount are the machine-vision standards: metal barrels, multi-element glass, iris and focus rings with lock screws, standardized flange distances (C: 17.53 mm; CS: 12.53 mm; a C lens fits a CS body via a 5 mm spacer, not conversely). Three separate data-quality channels:

1. *Sharpness (MTF) is the true floor under σ_d* : sub-pixel localization precision scales with the optical blur radius; a soft lens can double σ_d , and via (13) double σ_Z — equivalent to discarding half the sensor resolution (cf. the weakest-link law of subsection 10.4: the lens is a full member of that chain).
2. *Distortion modelability*: the Brown model (4) absorbs the smooth distortion of decent lenses; cheap wide M12s exhibit higher-order or non-radial terms that leave *systematic* calibration residuals.
3. *Stability of $\mathbf{K}$* : thread-focused plastic barrels creep with vibration and temperature — silent recalibration events, precisely what the $\mathbf{F}$ -drift monitor of section 6 exists to catch. Locked C/CS rings hold $\mathbf{K}$ for months. Autofocus is this failure mode as a feature: *a calibrated camera must never refocus*.

An iris adds one more useful knob: most lenses peak in sharpness around $f/4$ – $f/8$; beyond that diffraction itself blurs (spot $\approx 1.22 \lambda N$: at $f/8$ about $5.4 \mu\text{m}$, already exceeding a $3 \mu\text{m}$ pixel).

A.5 Monochrome vs color

A Bayer color sensor samples each color at $\frac{1}{4}$ – $\frac{1}{2}$ of the pixel grid and interpolates the rest; demosaicing smears exactly the high-contrast edges that corners and matching depend on, and costs about one stop of sensitivity. This pipeline processes grayscale throughout, so monochrome sensors deliver strictly better data; color remains at most a redundancy layer in the calibration pattern (subsection 5.6).

A.6 Summary and candidate hardware

Characteristic	Data-quality consequence	When it matters
rolling shutter	~ 0.5 px systematic shear (Example 19)	any motion
no trigger	phantom disparity, ~ 47 mm (Example 20)	dynamic scenes
UVC / MJPEG	structured artifacts in σ_d ; no trigger	beyond bring-up
M12 optics	MTF floor, distortion residuals, K creep	always, silently
C/CS optics	sharp, modelable, mechanically stable K	the metric rig
color (Bayer)	demosaic edge artifacts, -1 stop sensitivity	mono preferred

Table 4: Capture-hardware characteristics and their consequences.

Three procurement tiers follow (indicative 2026 prices):

1. *Bring-up* ($\sim \$150/\text{pair}$): two Logitech C920/C930-class UVC webcams. Rolling shutter, no trigger, but focus and exposure lockable via `v4l2-ctl`; teaches the pipeline every real-world lesson cheaply. Algorithms yes, metric claims no.
2. *Budget metric rig* ($\sim \$150$ – $250/\text{pair}$): OV9281-class modules (e.g. Arducam) — 1280×800 *monochrome global shutter*, external-trigger variants, interchangeable M12 optics (choose the best lens, lock it with thread-lock). With a 2.8–4 mm lens, $f \approx 950$ – 1300 px — the regime of all worked examples here. Same idea on CSI: the Raspberry Pi Global Shutter Camera (IMX296, trigger pin, $\sim \$50$).
3. *Solid metric rig* ($\sim \$500$ – $900/\text{pair}$): entry machine-vision USB3 (FLIR Blackfly S, Basler ace/dart, or Daheng/HIK equivalents): 1.6–2.3 MP mono global shutter, real trigger I/O, Linux SDKs, C/CS lenses. The tier where the resolution mathematics of subsection 10.3 becomes purchasable.

A packaged shortcut: the Luxonis OAK-D class devices ($\sim \$250$) contain two factory-synchronized global-shutter mono cameras in one USB3 housing — the whole hardware problem in one purchase, at the cost of a fixed ~ 7.5 cm baseline, short for a 4–6 m working volume ($\sigma_Z \propto 1/B$, (13)).

Table 5 lists concrete representatives, sorted by price; prices are indicative mid-2026 street prices per camera and drift, but the *characteristics* are fixed properties of each product.

Model	≈\$	Sensor / resolution	Mono	Global shutter	HW trigger	Raw frames	Lens
Raspberry Pi GS Camera ^a	50	IMX296, 1456 × 1088	— ^a	✓	✓	✓	CS
Logitech C920s ^b	70	1920 × 1080	—	—	—	— ^b	fixed
Arducam OV9281 USB	90	OV9281, 1280 × 800	✓	✓	✓	✓	M12
HIKrobot MV-CS016-10UM	230	IMX273, 1440 × 1080	✓	✓	✓	✓	C
Luxonis OAK-D ^c	250	2×OV9282, 1280 × 800	✓	✓	— ^c	✓	fixed
Basler dart daA1280-54um	280	1280 × 960	✓	✓	✓	✓	S/CS
Intel RealSense D435 ^d	330	2×OV9282 IR	✓	✓	— ^d	✓	fixed
FLIR Blackfly S 16S2M-CS	465	IMX273, 1440 × 1080	✓	✓	✓	✓	CS

Table 5: Representative commercial cameras, sorted by indicative 2026 price per unit. ^aCSI interface (needs a Raspberry Pi, not USB); color filter — monochrome variants from third parties. ^bUVC; uncompressed only at reduced rates, MJPEG otherwise; exposure and focus lockable via `v4l2-ctl`. ^cNo external trigger input, but the internal stereo pair is factory-synchronized — the trigger problem is solved *inside* the box; baseline fixed at 7.5 cm. ^dDepth device; the raw synchronized IR pair is accessible and usable as a stereo source; projector should be disabled for our matcher.

A.7 Blending heterogeneous cameras

Knowing each camera’s characteristics is itself an asset: a *modeled* defect is a correctable or at least gateable one. A rolling-shutter camera whose row-readout time is known can have its rows time-stamped individually and its observations gated out (or motion- compensated) whenever the scene moves; a camera with a measured noise curve enters the triangulation with an honest weight (subsection 10.4); a lens with well-fit distortion contributes unbiased rays while one with known residuals can be trusted only in its image center. The measurement model becomes per-camera, and every characteristic in Table 4 turns from a hidden bias into a term of that model.

A deliberate *blend* of camera types then does more than average — it makes artifacts detectable, because most artifacts are *camera-specific* while the scene is common. MJPEG block edges, rolling shear, demosaic fringes, and specular highlights each live in some cameras and not others (highlights move with viewpoint; compression artifacts sit on one camera’s grid); true structure must be consistent across all views. With identical cameras, a shared systematic error is invisible — every unit votes for the same lie; with heterogeneous hardware the correlation is broken and disagreement *localizes* the artifact, in the same spirit as the **F**-drift and laser-audit health monitors (section 6, subsection 11.2). Complementary pairings fall out naturally: a synchronized global-shutter mono pair as the metric backbone; a high-resolution color camera (rolling shutter tolerated, gated by motion) contributing fine texture and appearance to the model; a wide-FOV unit for coverage and moving-object flagging, its geometry weighted down per subsection 10.4. Geometry from the cameras built for geometry, appearance from the cameras built for appearance, and cross-checks from their disagreement — the *N*-camera analysis of section 10 applies unchanged, since every camera carries its own **K**, distortion, and noise model.

References

- [BM92] Paul J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992.

- [CL96] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH 96*, pages 303–312, 1996.
- [FB81] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [FP10] Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multiview stereopsis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(8):1362–1376, 2010.
- [FTV00] Andrea Fusiello, Emanuele Trucco, and Alessandro Verri. A compact algorithm for rectification of stereo pairs. *Machine Vision and Applications*, 12(1):16–22, 2000.
- [Gen11] Jason Geng. Structured-light 3d surface imaging: a tutorial. *Advances in Optics and Photonics*, 3(2):128–160, 2011.
- [GV13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013.
- [Har97] Richard I. Hartley. In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6):580–593, 1997.
- [Hor87] Berthold K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the Optical Society of America A*, 4(4):629–642, 1987.
- [HS97] Richard I. Hartley and Peter Sturm. Triangulation. *Computer Vision and Image Understanding*, 68(2):146–157, 1997.
- [HZ04] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, second edition, 2004.
- [ISM84] Seiji Inokuchi, Kosuke Sato, and Fumio Matsuda. Range imaging system for 3-d object recognition. In *Proceedings of the International Conference on Pattern Recognition*, pages 806–808, 1984.
- [KBH06] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, pages 61–70, 2006.
- [KS00] Kiriakos N. Kutulakos and Steven M. Seitz. A theory of shape by space carving. *International Journal of Computer Vision*, 38(3):199–218, 2000.
- [Lau94] Aldo Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2):150–162, 1994.
- [LH81] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293:133–135, 1981.
- [Low04] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

- [OK93] Masatoshi Okutomi and Takeo Kanade. A multiple-baseline stereo. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(4):353–363, 1993.
- [SCD⁺06] Steven M. Seitz, Brian Curless, James Diebel, Daniel Scharstein, and Richard Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 519–528, 2006.
- [SM99] Peter F. Sturm and Stephen J. Maybank. On plane-based camera calibration: A general algorithm, singularities, applications. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 432–437, 1999.
- [SS02] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1–3):7–42, 2002.
- [SSS06] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: Exploring photo collections in 3d. *ACM Transactions on Graphics*, 25(3):835–846, 2006.
- [TMHF00] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment — a modern synthesis. In *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372. Springer, 2000.
- [Zha00] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.